Results Report: WT vs pifq single-cell RNA-seq pipeline

Executed results — real figures from a fresh run using the bibliography annotation (celltype); code shown as in the methods paper

SingleCell Pipeline

2026-07-28

Contents

Minimum computing requirements . . . 2

1 Part 1 - Environment and raw data 3
1.1 Clone the repository . . . 3
1.2 Container startup (Docker or Singularity) . . . 3
1.3 Raw data download and Cell Ranger processing . . . 4

2 Part 2 - Single-cell analysis in R 6
2.1 Configuration . . . 6
2.2 Initialization . . . 6
2.3 Section 1 - Data loading and pre-filter QC . . . 7
2.4 Section 2 - Cell filtering and doublet detection . . . 8
2.5 Section 3 - Merge and initial preprocessing . . . 9
2.6 Section 4 - Harmony batch correction . . . 10
2.7 Section 5 - Resolution optimization . . . 11
2.8 Section 6 - Final clustering . . . 12
2.9 Section 7 - Cell-type annotation . . . 13
2.10 Section 8 - Annotated clustree . . . 14
2.11 Section 9 - Gene expression visualization . . . 15
2.12 Section 10 - Cell-type grouping [optional] . . . 16
2.13 Section 11 - Subcluster marker inspection [optional] . . . 17
2.14 Section 12 - Export the curated object . . . 17

3 Part 3 - Pseudobulk differential expression and GO 19
3.1 Section 13 - Cell-type subsets . . . 19
3.2 Section 14 - Pseudo-replicate assignment . . . 19
3.3 Section 15 - Pseudobulk tables and DESeq2 . . . 20
3.4 Section 16 - Volcano plots . . . 20
3.5 Section 17 - Differential gene tables . . . 21
3.6 Section 18 - GO enrichment . . . 22
3.7 Section 19 - Log2FC heatmap . . . 22
3.8 Section 20 - Gene-gene coexpression and TF network . . . 24

4 Part 4 - Pseudotime trajectory analysis (Python) 25
4.1 Section 23 - Setup . . . 25
4.2 Section 24 - Load curated object . . . 25
4.3 Section 25 - Cell type selection . . . 25
4.4 Section 26 - Trajectory inference . . . 26

4.5 Section 27 - Plot genes on trajectory . . . 27
4.6 Section 28 - Gene trends . . . 27
4.7 Section 29 - Final export . . . 29

Supplementary Table — Software dependencies and versions **30**
Interpreters and command-line tools . . . 30
R packages (421) . . . 30
Python packages (207) . . . 33

This methods-paper R Markdown documents the full WT vs pifq workflow: two samples, `WT` and `pifq`, processed and integrated together, then carried through three chapters — single-cell preprocessing, clustering and annotation (Part 2); pseudobulk differential expression, GO and the co-expression network (Part 3); and pseudotime trajectory analysis (Part 4). The code blocks are shown exactly as in the methods paper (not executed inline); the figures below are the **real outputs** of a fresh run of `workflow/step1_singlecell.R`, `workflow/step2_degs.R` and `workflow/step3_pseudotime.py`, written to a separate `resultados_rerun/` directory using the bibliography annotation (`celltype`).

The active annotation path is bibliography-based (`biblio_marks.txt` -> `celltype_curated`). Reference-transfer annotation is not part of this draft.

Minimum computing requirements

The table below summarizes the computing requirements for running the full pipeline on a personal computer, from raw FASTQ through the downstream Seurat/R and Python workflow. These are not hard limits; below them the pipeline still runs, just slower, and heavier steps (Cell Ranger, merging, PCA/UMAP) risk stalling or exhausting memory.

Component	Minimum	Recommended
CPU cores	4	16 or more
RAM	32 GB	64-128 GB for merged multi-sample objects
Disk space	80 GB free	100 GB free
Container runtime	Any OCI-compatible runtime (e.g. Docker)	Docker or Singularity/Apptainer

This workflow is packaged as a single public image (`matigara/scrnaseq:latest`, on Docker Hub); the `docker-compose` YAML used to build and run it is kept in the repository. Any container runtime able to run that image works; the setup below covers both Docker and Singularity/Apptainer (the latter common on HPC clusters, where Docker is usually not permitted).

Small pilot datasets can be run on a laptop or desktop if memory is sufficient. For full experiments, repeated clustering sweeps, Cell Ranger processing from FASTQ, or large merged Seurat objects, a server or external cluster is strongly recommended. Cell Ranger is CPU- and memory-intensive and is not installed inside the downstream `matigara/scrnaseq:latest` Docker image.

1 Part 1 - Environment and raw data

1.1 Clone the repository

Choose or create a working directory, then clone the pipeline repository into it:

Bash

```bash
mkdir -p ~/my-analysis
cd ~/my-analysis
git clone https://github.com/cliford2001/SingleCell.git
cd SingleCell
```

All paths referenced later in this document (`workflow/`) are relative to this cloned repository. Its layout is:

```
SingleCell/
|-- Dockerfile
|-- docker-compose.yml
|-- README.md
|-- data/
|   |-- AtTFDB_loci.txt
|   |-- biblio_marks.txt
|   |-- Arabidopsis_thaliana_TAIR10.1_Araport11_genome.fasta.gz
|   `-- Arabidopsis_thaliana_TAIR10.1_Araport11_annotation.gtf.gz
`-- workflow/
    |-- load_libraries.R                # R packages
    |-- load_libraries_python.py        # Python packages
    |-- ScRNA_Analysis_Functions.R      # helper functions (Steps 1-2)
    |-- ScRNA_Pseudotime_Functions.py   # helper functions (Step 3)
    |-- step0_cellranger.sh             # Part 1 - Cell Ranger (bash)
    |-- step1_singlecell.R              # Part 2 - single-cell analysis
    |-- step2_degs.R                    # Part 3 - pseudobulk DE + GO
    `-- step3_pseudotime.py             # Part 4 - pseudotime (Python)
```

The bibliography marker table (`data/biblio_marks.txt`) is part of the repository for this WT vs pifq run (see Section 1).

1.2 Container startup (Docker or Singularity)

The container mounts the cloned repository itself as `/workspace`, so results are written back into the repository (`resultados/`). Since results are not pushed upstream, this is not a problem. The same public image runs under either runtime; always pull it rather than rebuilding from the `Dockerfile`, so every user gets the exact tested package versions.

Docker. Mount the cloned repository and open a shell:

Bash

```bash
docker run -it --rm \
  -v $(pwd):/workspace \
  matigara/scrnaseq:latest /bin/bash
```

Or, from inside the cloned repository, `docker compose run r` uses the bundled `docker-compose.yml` (same image, same `/workspace` mount). On the current server the Docker container is already running and the project is mounted at `/workspace`.

Singularity / Apptainer. No root is required. Convert the public Docker Hub image to a local `.sif` once, then open a shell binding the repository to `/workspace`:

Bash

```
singularity build scrnaseq.sif docker://matigara/scrnaseq:latest
singularity exec --bind $(pwd):/workspace scrnaseq.sif /bin/bash
```

Apptainer imports the image environment, so the bundled Python (`/opt/venv`) and R stay on `PATH`, and it runs as the invoking user, so files under `resultados/` are owned by that user rather than by root.

1.3 Raw data download and Cell Ranger processing

Run this whole section outside the Docker container, on a host, server, or HPC where Cell Ranger is installed. Cell Ranger is *not* bundled inside the `matigara/scrnaseq:latest` image (that image only covers the downstream R / Python workflow), so running `cellranger` inside the container returns `command not found`. Write the Cell Ranger outputs into the cloned repository directory so they are visible from inside the container (which mounts that same directory as `/workspace`).

Install Cell Ranger 9.0.1 (proprietary 10x Genomics software; this workflow uses version 9.0.1). Get it from the 10x Genomics *previous versions* page: https://www.10xgenomics.com/support/software/cell-ranger/downloads/previous-versions

1. Log in and accept the license.
2. Find **Cell Ranger 9.0.1** and copy its `curl` command.
3. That URL is signed and time-limited (it carries an `Expires=` timestamp tied to your account), so paste the current one — it cannot be hard-coded here or reused later. A plain `git clone` of the source repo does *not* give a ready-to-run binary.

Then extract it and add it to your `PATH`:

Bash

```
# Paste the signed curl command for 9.0.1 from the 10x page, e.g.:
curl -o cellranger-9.0.1.tar.gz "https://cf.10xgenomics.com/releases/cell-exp/\
cellranger-9.0.1.tar.gz?Expires=...&Key-Pair-Id=...&Signature=..."

tar -xzvf cellranger-9.0.1.tar.gz
export PATH=$PWD/cellranger-9.0.1/bin:$PATH
cellranger --version
```

The single-cell workflow starts from Cell Ranger output, specifically the `outs/filtered_feature_bc_matrix/` folder for each sample. If only raw FASTQ files are available, first download or stage the FASTQs, then run Cell Ranger for each sample before starting the R workflow.

The WT vs pifq FASTQ files used in this run are public data from Han et al., 2023, *Nature Plants*, deposited at the CNCB Genome Sequence Archive under BioProject PRJCA016521, run accession `CRA010863`:

Sample	Run accession	Files
ScWT	CRR775298	`CRR775298_f1.fastq.gz`, `CRR775298_r2.fastq.gz`
Scpifq	CRR775297	`CRR775297_f1.fastq.gz`, `CRR775297_r2.fastq.gz`

Download the reads, then rename/symlink them to the naming Cell Ranger expects (`<sample>_S1_L001_R1/R2_001.fastq.gz`)

Bash

```
mkdir -p /workspace/fastq/CRA010863

wget -c -P /workspace/fastq/CRA010863 \
  ftp://download.big.ac.cn/gsa2/CRA010863/CRR775298/CRR775298_f1.fastq.gz
wget -c -P /workspace/fastq/CRA010863 \
  ftp://download.big.ac.cn/gsa2/CRA010863/CRR775298/CRR775298_r2.fastq.gz
wget -c -P /workspace/fastq/CRA010863 \
  ftp://download.big.ac.cn/gsa2/CRA010863/CRR775297/CRR775297_f1.fastq.gz
wget -c -P /workspace/fastq/CRA010863 \
  ftp://download.big.ac.cn/gsa2/CRA010863/CRR775297/CRR775297_r2.fastq.gz

cd /workspace/fastq/CRA010863
ln -s CRR775298_f1.fastq.gz ScWT_S1_L001_R1_001.fastq.gz
ln -s CRR775298_r2.fastq.gz ScWT_S1_L001_R2_001.fastq.gz
ln -s CRR775297_f1.fastq.gz Scpifq_S1_L001_R1_001.fastq.gz
ln -s CRR775297_r2.fastq.gz Scpifq_S1_L001_R2_001.fastq.gz
```

After FASTQs are available, run Cell Ranger for each sample. Cell Ranger itself is not redistributed in this repository; download it from the official 10x Genomics GitHub release (https://github.com/10XGenomics/cellranger/releases/tag/cellranger-9.0.1) and add it to your `PATH`. Run all commands in this section from the cloned repository directory, on a host/server/HPC where Cell Ranger is installed (not inside the container).

The reference genome (TAIR10.1) and Araport11 annotation ship compressed in the repository under `data/`, so no download is needed. Decompress them and build the Cell Ranger reference:

Bash

```
gunzip -k data/Arabidopsis_thaliana_TAIR10.1_Araport11_genome.fasta.gz
gunzip -k data/Arabidopsis_thaliana_TAIR10.1_Araport11_annotation.gtf.gz

cellranger mkref \
  --genome=Ara \
  --fasta=data/Arabidopsis_thaliana_TAIR10.1_Araport11_genome.fasta \
  --genes=data/Arabidopsis_thaliana_TAIR10.1_Araport11_annotation.gtf
```

Run Cell Ranger counts for each sample:

Bash

```
cellranger count --localcores=80 --id=ScWT --fastqs=/workspace/fastq/CRA010863 \
  --sample=ScWT --transcriptome=Ara --create-bam false

cellranger count --localcores=80 --id=Scpifq --fastqs=/workspace/fastq/CRA010863 \
  --sample=Scpifq --transcriptome=Ara --create-bam false
```

Use the resulting `filtered_feature_bc_matrix/` path in the sample manifest in Section 1.

2 Part 2 - Single-cell analysis in R

From this point on, the pipeline runs in R inside the Docker container. Start R:

Bash

```
R
```

The next two steps, Configuration and Initialization, set up the paths and helper functions used by every section that follows.

2.1 Configuration

Edit this block before running the pipeline.

R

```r
PIPELINE_DIR <- "/workspace/workflow"
DATA_DIR   <- "/workspace/."
base_dir   <- file.path(DATA_DIR, "resultados")
```

2.2 Initialization

Load helper scripts, set reproducibility options, and create output directories.

R

```r
source(file.path(PIPELINE_DIR, "load_libraries.R"))
source(file.path(PIPELINE_DIR, "ScRNA_Analysis_Functions.R"))
set.seed(1807)
options(Seurat.allow.s4 = FALSE)
setwd(DATA_DIR)
list2env(create_pipeline_dirs(base_dir), envir = .GlobalEnv)
output_dir <- base_dir
```

2.3 Section 1 - Data loading and pre-filter QC

R

```
output_dir <- dir_01
samples <- list(
  list(
    file      = "ScWT/outs/filtered_feature_bc_matrix",
    label     = "WT",
    condition = "WT"
  ),
  list(
    file      = "Scpifq/outs/filtered_feature_bc_matrix",
    label     = "pifq",
    condition = "pifq"
  )
)
colors <- c(
  "WT"  = "#66c2a5",
  "pifq"  = "#fc8d62"
)
mt_pattern <- "^ATMG"
cp_pattern <- "^ATCG"
seurat_list_raw <- load_seurat_samples(samples = samples,
                                       DATA_DIR = DATA_DIR,
                                       mt_pattern = mt_pattern,
                                       cp_pattern = cp_pattern)
plot_qc_batch(seurat_list_raw, colors, "qc_prefilter.pdf")
message("\nOK SECTION 1 COMPLETE: QC pre-filter plots saved")
```

Representative output from Section 1:

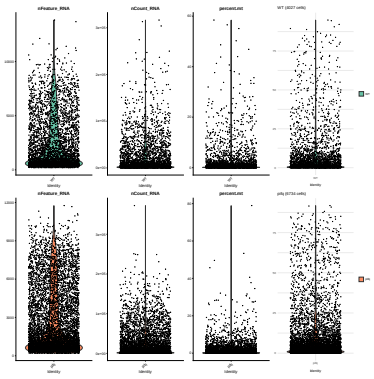


Figure 1: Pre-filter QC violin plots.

2.4 Section 2 - Cell filtering and doublet detection

R

```
output_dir <- dir_01
seurat_list <- filter_seurat_samples(seurat_list_raw, min_features = 200, max_mt = 5)
plot_qc_batch(seurat_list, colors, "qc_postfilter.pdf")
saveRDS(seurat_list, file.path(dir_objects, "seurat_list_postfilter.rds"))
message("\nOK SECTION 2 COMPLETE: filtering and doublet detection complete")
```

Representative output from Section 2:

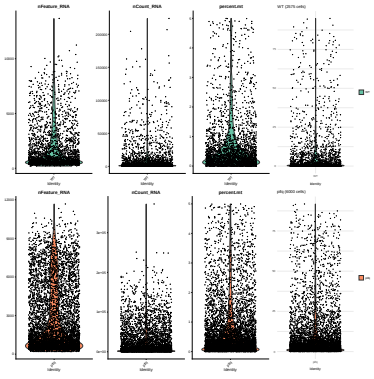


Figure 2: Post-filter QC violin plots.

2.5 Section 3 - Merge and initial preprocessing

R

```
output_dir <- dir_01
pbmc_harmony <- reduce(seurat_list, merge) %>%
  NormalizeData(verbose = FALSE) %>%
  FindVariableFeatures(
    selection.method = "vst", nfeatures = 2000, verbose = FALSE
  ) %>%
  ScaleData(verbose = FALSE) %>%
  RunPCA(npcs = 30, verbose = FALSE) %>%
  RunUMAP(reduction = "pca", dims = 1:30, verbose = FALSE)
save_pdf(
  DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),
  "umap_preharmony.pdf"
)
saveRDS(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_preharmony.rds"))
message("\nOK SECTION 3 COMPLETE: merge and preprocessing complete")
```

Representative output from Section 3:

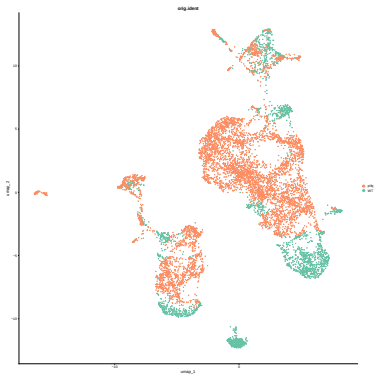


Figure 3: UMAP before Harmony correction, colored by sample.

2.6 Section 4 - Harmony batch correction

R

```
output_dir <- dir_01
pbmc_harmony <- pbmc_harmony %>%
  RunHarmony("orig.ident", plot_convergence = FALSE) %>%
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE)
save_pdf(
  DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),
  "umap_postharmony.pdf"
)
saveRDS(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_postharmony.rds"))
message("\nOK SECTION 4 COMPLETE: Harmony integration complete")
```

Representative output from Section 4:

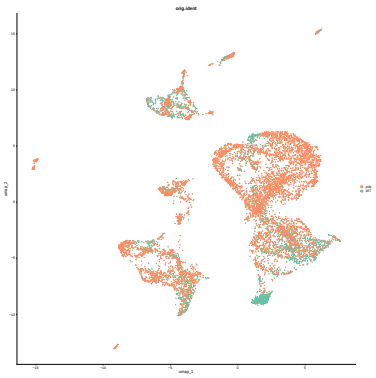


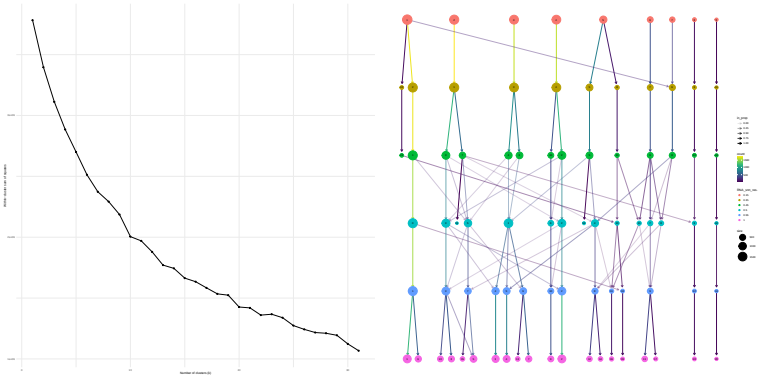
Figure 4: UMAP after Harmony correction, colored by sample.

2.7 Section 5 - Resolution optimization

R

```
resolutions_test <- c(0.15, 0.35, 0.45, 0.55, 1.0)
output_dir <- dir_02
k_range <- 1:31
pca_data <- Embeddings(pbmc_harmony, "pca")[, 1:30]
wss <- sapply(
  k_range,
  function(k) kmeans(pca_data, centers = k, nstart = 4)$tot.withinss
)
elbow_plot <- ggplot(data.frame(k = k_range, wss = wss), aes(k, wss)) +
  geom_line() +
  geom_point() +
  labs(
    x = "Number of clusters (k)",
    y = "Within-cluster sum of squares"
  ) +
  theme_minimal()
save_pdf(elbow_plot, "elbow_plot.pdf", w = 18, h = 18)
clu <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE) %>%
  FindNeighbors(reduction = "harmony", dims = 1:30, k.param = 20, verbose = FALSE)
for (res in resolutions_test)
  clu <- FindClusters(clu, resolution = res, algorithm = 4, verbose = FALSE)
save_pdf(clustree(clu, prefix = "RNA_snn_res."), "clustree2.pdf", w = 18, h = 18)
message("\nOK SECTION 5 COMPLETE: elbow plot and clustree saved")
```

Representative outputs from Section 5:



2.8 Section 6 - Final clustering

R

```
cluster_resolution <- 0.35
output_dir <- dir_02
pbmc_harmony <- pbmc_harmony %>%
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE) %>%
  FindNeighbors(reduction = "harmony", dims = 1:30, k.param = 20, verbose = FALSE) %>%
  FindClusters(resolution = cluster_resolution, algorithm = 4, verbose = FALSE)
Idents(pbmc_harmony) <- "seurat_clusters"
save_pdf(
  DimPlot(pbmc_harmony, group.by = "seurat_clusters", label = TRUE),
  "umap_seuratclusters.pdf"
)
message("\nOK SECTION 6 COMPLETE: final clustering complete")
```

Representative output from Section 6:

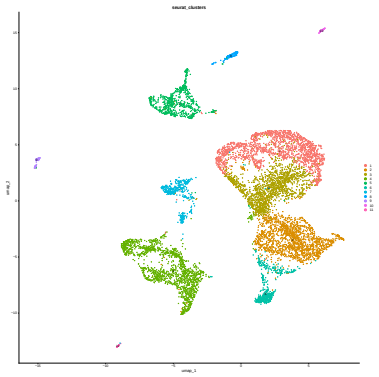


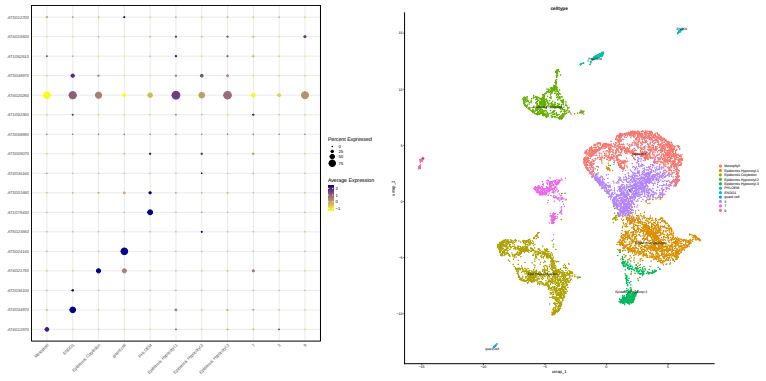
Figure 5: Final UMAP colored by numbered Seurat clusters.

2.9 Section 7 - Cell-type annotation

R

```
output_dir <- dir_03
biblio_marks_file <- file.path(DATA_DIR, "data", "biblio_marks.txt")
marker_table <- read.table(
  biblio_marks_file, header = TRUE, sep = "\t", quote = ""
)
markers <- find_markers(
  pbmc_harmony,
  output_file = file.path(output_dir, "FindAllMarkers.tsv")
)
pbmc_harmony <- annotate_by_markers(
  pbmc_harmony, markers,
  reference_file = biblio_marks_file
)
plot_marker_dotplot(
  pbmc_harmony,
  marker_table,
  annot_col = "celltype",
  outfile   = file.path(
    output_dir, "dotplot_marker_table_annotation_biblio.pdf"
  ),
  width = 18, height = 18
)
save_pdf(
  DimPlot(
    pbmc_harmony, group.by = "celltype",
    label = TRUE, repel = TRUE, raster = FALSE
  ),
  "umap_annotation_biblio.pdf"
)
message("\nOK SECTION 7 COMPLETE: cell-type annotation complete")
```

Representative outputs from Section 7:



2.10 Section 8 - Annotated clustree

R

```
output_dir <- dir_03
Mode <- function(x) {
  x <- as.character(x)
  x <- x[!is.na(x) & nzchar(x)]
  if (length(x) == 0) return(NA_character_)
  names(sort(table(x), decreasing = TRUE))[1]
}
stopifnot(exists("clu"))
stopifnot("celltype" %in% colnames(pbmc_harmony@meta.data))
celltype_label <- as.character(pbmc_harmony$celltype)
names(celltype_label) <- Cells(pbmc_harmony)
clu$celltype_label <- celltype_label[Cells(clu)]
print(table(clu$celltype_label, useNA = "ifany"))
save_pdf(
  clustree(
    clu, prefix = "RNA_snn_res.",
    node_label = "celltype_label", node_label_aggr = "Mode"
  ),
  "clustree_annotated.pdf", w = 14, h = 14
)
message("\nOK SECTION 8 COMPLETE: annotated clustree saved")
```

Representative output from Section 8:

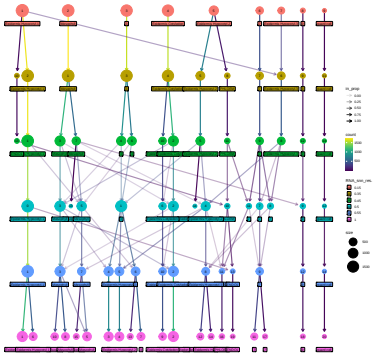


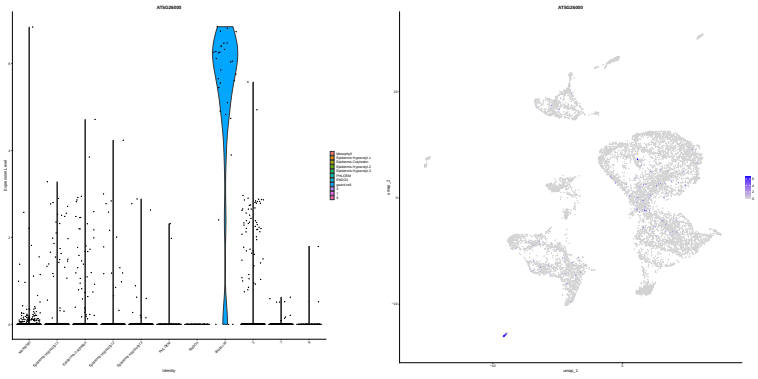
Figure 6: Cluster tree labeled by the dominant bibliography-based annotation in each node.

2.11 Section 9 - Gene expression visualization

R

```
gene <- "AT5G26000"
output_dir <- dir_04
pbmc_harmony <- JoinLayers(pbmc_harmony)
Idents(pbmc_harmony) <- "celltype"
save_vln(
  VlnPlot(pbmc_harmony, features = gene),
  "vln_gene_all.pdf"
)
save_pdf(
  FeaturePlot(pbmc_harmony, features = gene),
  "feature_gene_all.pdf"
)
message("\nOK SECTION 9 COMPLETE: gene expression visualization saved")
```

Representative outputs from Section 9:



2.12 Section 10 - Cell-type grouping [optional]

R

```r
grouping <- c()
# Example:
# grouping <- c(
#   "Epidermis Hypocotyl.1" = "Epidermis Hypocotyl",
#   "Epidermis Hypocotyl.2" = "Epidermis Hypocotyl"
# )

output_dir <- dir_05

if (length(grouping) > 0) {
  pbmc_harmony$celltype_grouped <- recode(pbmc_harmony$celltype, !!!grouping)
} else {
  pbmc_harmony$celltype_grouped <- pbmc_harmony$celltype
}

save_pdf(
  DimPlot(
    pbmc_harmony, group.by = "celltype_grouped",
    label = TRUE, repel = TRUE, raster = FALSE
  ),
  "umap_grouped.pdf"
)

message("\nOK SECTION 10 COMPLETE: cell-type grouping complete")
```

Representative output from Section 10:

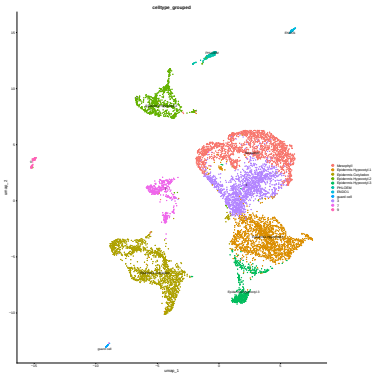


Figure 7: UMAP colored by grouped cell-type labels.

2.13 Section 11 - Subcluster marker inspection [optional]

This section is intentionally interactive. `inspect_subcluster_markers()` subclusters one cluster and saves a combined figure (subcluster UMAP, marker dotplot, and a small FeaturePlot grid) to help decide whether curation is needed. After reviewing the figures, `curate_clusters()` re-inspects the chosen clusters and reassigns their subclusters to the labels you provide, writing the result into `celltype_curated`. Both helpers live in `ScRNA_Analysis_Functions.R`.

R

```r
output_dir <- dir_05
Idents(pbmc_harmony) <- "celltype"

# 1. Inspect a cluster (saves the combined subcluster + marker figure)
inspect_subcluster_markers(
  pbmc_harmony, cluster_id = "2",
  marker_table = marker_table, output_dir = output_dir
)

# 2. Map each cluster's subclusters to labels ("others" = any ID not listed),
#    then curate them in one call. Example for two clusters:
reassign <- list(
  "2" = c("0" = "Identity A", "others" = "Unresolved"),
  "4" = c("1" = "Identity B", "others" = "Unresolved")
)
pbmc_harmony <- curate_clusters(
  pbmc_harmony, reassign,
  marker_table = marker_table, output_dir = output_dir
)

# 3. Save the curated annotation UMAP
save_pdf(
  DimPlot(pbmc_harmony, group.by = "celltype_curated",
          label = TRUE, repel = TRUE, raster = FALSE),
  "umap_curated.pdf"
)

message("\nOK SECTION 11 COMPLETE: curation complete")
```

Section 11 (interactive subcluster curation) was not re-executed in this run. All downstream analyses use the bibliography annotation (`celltype`) directly, so unresolved clusters keep their numeric labels (2, 4, 5, 7).

2.14 Section 12 - Export the curated object

Closes Chapter 1 by saving the curated object in two formats: an `.rds` checkpoint for the R-based Chapter 2, and an AnnData `.h5ad` for the Python-based Chapter 3 trajectory and velocity analyses (Scanpy, scFates, Palantir - all pre-installed in the Docker image).

R

```r
# Checkpoint - restore with:
#   pbmc_harmony <- readRDS(file.path(dir_objects, "pbmc_harmony_curated.rds"))
saveRDS(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_curated.rds"))

# Export the curated object to AnnData h5ad format for Python-based
# trajectory and velocity analyses.
export_to_scanpy(
  pbmc_harmony,
  file.path(dir_objects, "pbmc_harmony_curated.h5ad")
)

# To export a specific cell type only:
# export_to_scanpy(
#   subset(pbmc_harmony, subset = celltype_curated == "guard cell"),
#   file.path(dir_objects, "GuardCell.h5ad")
# )

message("\nOK SECTION 12 COMPLETE: curated object saved (.rds + .h5ad)")
```

3 Part 3 - Pseudobulk differential expression and GO

This part continues from the object saved at the end of Chapter 1. It aggregates cells into pseudo-replicates per cell type, runs pseudobulk DESeq2 for the WT vs pifq contrast, and summarizes results with volcano plots, differential tables, GO enrichment, a log2FC heatmap, and co-expression networks.

R

```
pbmc_harmony <- readRDS(file.path(dir_objects, "pbmc_harmony_curated.rds"))
```

3.1 Section 13 - Cell-type subsets

The annotation column used to define cell types is a user choice. Depending on how Chapter 1 was run, this is one of `celltype` (bibliography annotation), `celltype_grouped` (broad groups from Section 10), or `celltype_curated` (manual curation from Section 11). All downstream sections in this part use whichever column is set here.

R

```
pseudobulk_annot_col <- "celltype_curated"
table(pbmc_harmony[[pseudobulk_annot_col]])
cell_type_subsets <- create_cell_type_subsets(
  pbmc_harmony, annot_col = pseudobulk_annot_col
)
message("\nOK SECTION 13 COMPLETE: cell-type subsets created")
```

Cells per cell type (first three shown):

Cell type	Cells
Epidermis_Hypocotyl_1	1847
Epidermis_Cotyledon	1461
Mesophyll	488

3.2 Section 14 - Pseudo-replicate assignment

R

```
pseudobulk_conditions <- NULL
n_pseudoreps          <- 3
cell_type_subsets_replicates <- assign_pseudoreplicates_batch(
  cell_type_subsets,
  pseudobulk_conditions = pseudobulk_conditions,
  n_pseudoreps          = n_pseudoreps
)
table(cell_type_subsets_replicates[["Mesophyll"]]$replicate)
message("\nOK SECTION 14 COMPLETE: pseudo-replicates assigned")
```

Cell types present in only one condition (e.g. `C2`, present in only 57 cells from a single sample) are dropped here, since a WT vs pifq contrast needs both conditions represented.

Cells per pseudo-replicate (first three cell types shown):

Cell type	Replicate	Cells
Epidermis_Hypocotyl_1	pifq_rep1	329
Epidermis_Hypocotyl_1	pifq_rep2	366
Epidermis_Hypocotyl_1	pifq_rep3	346
Epidermis_Hypocotyl_1	WT_rep1	279
Epidermis_Hypocotyl_1	WT_rep2	272
Epidermis_Hypocotyl_1	WT_rep3	255
Epidermis_Cotyledon	pifq_rep1	316
...	...	...

3.3 Section 15 - Pseudobulk tables and DESeq2

R

```r
comparisons <- list(
  list(conds = c("WT", "pifq"), tag = "WT_vs_pifq")
)
cell_types_to_analyze <- NULL
output_dir <- dir_06
deseq2_results <- run_pseudobulk_deseq2_analysis(
  cell_type_subsets_replicates = cell_type_subsets_replicates,
  comparisons    = comparisons,
  output_dir     = output_dir,
  cell_types     = cell_types_to_analyze,
  pseudobulk_dir = file.path(dir_objects, "pseudobulk_replicas")
)
message("\nOK SECTION 15 COMPLETE: pseudobulk aggregation and DESeq2 complete")
```

One `DESeq2_<cell_type>_WT_vs_pifq.csv` is written per cell type: the standard DESeq2 results table (one row per gene, with `log2FoldChange`, `pvalue`, and `padj`). Positive log2FC means higher expression in pifq. First rows of the Mesophyll table:

	baseMean	log2FoldChange	lfcSE	stat	pvalue	padj
AT1G01010	46.22	-2.102	0.307	-6.857	7.0e-12	9.7e-11
AT1G01020	48.78	0.100	0.326	0.306	7.6e-01	8.5e-01
AT1G01030	172.27	0.437	0.306	1.428	1.5e-01	2.8e-01

3.4 Section 16 - Volcano plots

R

```r
volcano_tag <- "WT_vs_pifq"
padj_cut    <- 0.05
lfc_cut     <- 1
render_volcano_plots(
  results_dir = file.path(dir_06, volcano_tag),
  output_dir  = file.path(dir_06, volcano_tag, "volcano"),
  pdf_name    = paste0("VolcanoPlots_", volcano_tag, ".pdf"),
  padj_cut    = padj_cut,
  lfc_cut     = lfc_cut
)
message("\nOK SECTION 16 COMPLETE: volcano plots saved")
```

Representative output from Section 16:

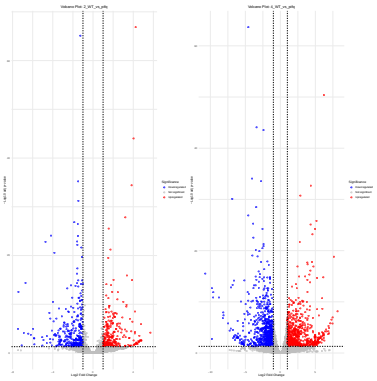


Figure 8: Volcano plot for Mesophyll, WT vs pifq.

3.5 Section 17 - Differential gene tables

R

```r
diff_prefix <- paste0("diff_table_", volcano_tag)
diff_tables <- build_differential_tables(
  results_dir = file.path(dir_06, volcano_tag),
  output_dir  = file.path(dir_06, volcano_tag),
  padj_cut    = padj_cut,
  lfc_cut     = lfc_cut,
  prefix      = diff_prefix
)
message("\nOK SECTION 17 COMPLETE: differential gene tables saved")
```

This writes two combined matrices (genes x cell types). The class matrix (`diff_table_WT_vs_pifq_*.tsv`) uses `+1/-1/0` to flag genes up-, down-, or not significantly regulated in each cell type; the log2FC matrix (`tabla_log2FC_*.tsv`) holds the actual fold changes (`NA` where not significant). Example rows for three annotated cell types (Epidermis Cotyledon, guard cell, Mesophyll):

```
# class matrix                          # log2FC matrix
gene_id    EpiCot  guard  Meso         gene_id    EpiCot  guard  Meso
AT1G29930    1       1      1          AT1G29930    4.79    3.51   2.02
```

```
AT1G54410   -1       1     -1        AT1G54410   -1.23    2.52  -1.88
AT1G55670    1       1      1        AT1G55670    3.58    4.05   1.66
```

3.6 Section 18 - GO enrichment

R

```r
go_space    <- "BP"
go_orgdb    <- org.At.tair.db
go_keytype  <- "TAIR"
padj_cutoff <- 0.05
go_results <- run_go_enrichment_for_contrast(
  results_dir  = file.path(dir_06, volcano_tag),
  output_dir   = file.path(dir_07, volcano_tag),
  orgdb        = go_orgdb,
  keytype      = go_keytype,
  go_space     = go_space,
  padj_cutoff  = padj_cutoff,
  contrast_tag = volcano_tag
)
message("\nOK SECTION 18 COMPLETE: GO enrichment complete")
```

Representative output from Section 18:

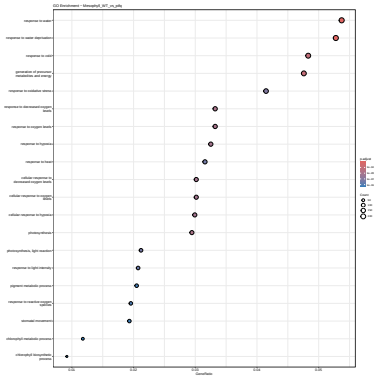


Figure 9: GO enrichment (BP) for Mesophyll, WT vs pifq.

3.7 Section 19 - Log2FC heatmap

Rows (genes) are ordered by hierarchical clustering, so genes with similar cross-cell-type profiles group together. Columns (cell types) follow the marker-table order via `cell_order`; cell types not present in the marker table (the unannotated clusters) are placed at the end.

R

```r
heatmap_limits <- c(-5, 5)
build_logfc_heatmap(
  logfc_table  = diff_tables$logfc,
  contrast_tag = volcano_tag,
  output_dir   = file.path(dir_06, volcano_tag),
  limits       = heatmap_limits,
  cell_order   = unique(marker_table$cell.types)
)
message("\nOK SECTION 19 COMPLETE: log2FC heatmap saved")
```

Representative output from Section 19:

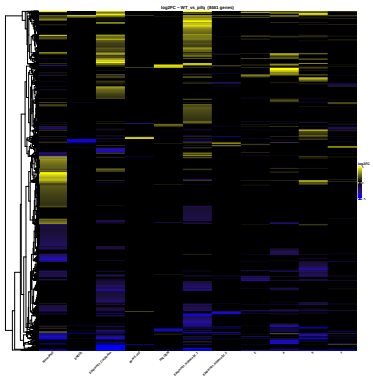


Figure 10: log2FC heatmap: rows hierarchically clustered, columns in marker-table order (unannotated clusters last), WT vs pifq.

3.8 Section 20 - Gene-gene coexpression and TF network

`run_tf_coexpression_network()` does the whole network step in one call: it builds the hdWGCNA gene-gene co-expression network from the significant DE genes, exports the edge list (`edges_unified.tsv`, `source | target | weight`), then filters to TF-containing pairs (AtTFDB) and draws the TF network colored by DE direction. No module-clustering plots are produced. Only the parameters below are meant to be edited.

R

```
run_tf_coexpression_network(
  seurat_obj      = pbmc_harmony,
  de_table_path   = file.path(dir_06, volcano_tag, "tabla_log2FC_fc1_padj_005.tsv"),
  tf_list_path    = file.path(DATA_DIR, "data", "AtTFDB_loci.txt"),
  output_dir      = dir_08,
  annot_col       = pseudobulk_annot_col,
  contrast_tag    = volcano_tag,
  n_metacells     = 50,
  min_module_size = 20,
  deep_split      = 2,
  soft_power      = NULL,
  tom_threshold   = 0.2,
  n_hub_label     = 15
)
message("\nOK SECTION 20 COMPLETE: TF coexpression network saved")
```

Representative output from Section 20:

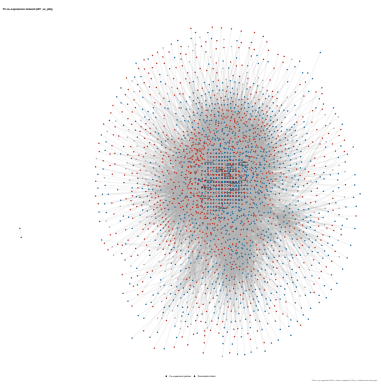


Figure 11: TF co-expression network, colored by DE direction.

4 Part 4 - Pseudotime trajectory analysis (Python)

Chapter 3 runs in Python (scanpy, Palantir, scFates), starting from the h5ad object exported at the end of Chapter 1 (`resultados/objects/pbmc_harmony_curated.h5ad`). Exit R and start Python inside the same Docker container:

Bash

```
q()           # leave R (do not save the workspace)
python3
```

4.1 Section 23 - Setup

Python

```
import os
PIPELINE_DIR = "/workspace/workflow"
DATA_DIR     = "/workspace/."
base_dir     = os.path.join(DATA_DIR, "resultados")
exec(open(os.path.join(PIPELINE_DIR, "load_libraries_python.py")).read(), globals())
exec(open(os.path.join(PIPELINE_DIR, "ScRNA_Pseudotime_Functions.py")).read(), globals())
dir_pseudotime = os.path.join(base_dir, "09_pseudotime")
os.makedirs(dir_pseudotime, exist_ok=True)
print("SECTION 23 COMPLETE: setup done")
```

4.2 Section 24 - Load curated object

Reads the AnnData object, converts the Seurat-style UMAP/PCA/HARMONY coordinate names to scanpy names, saves an overview UMAP, and prints the available cell types (used to choose the trajectory in Section 25).

Python

```
INPUT_H5AD     = "/workspace/resultados/objects/pbmc_harmony_curated.h5ad"
ANNOTATION_COL = "celltype_curated"
N_JOBS         = 4
adata, N_JOBS = load_curated_object(
    input_h5ad     = INPUT_H5AD,
    dir_pseudotime = dir_pseudotime,
    annotation_col = ANNOTATION_COL,
    n_jobs         = N_JOBS,
)
```

Representative output from Section 24:

4.3 Section 25 - Cell type selection

Picks the cell types used to build the trajectory. Here we follow the stomatal lineage: cotyledon epidermis differentiating into guard cells. Names must match the Section 24 list exactly.

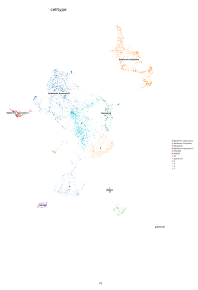


Figure 12: Overview UMAP colored by curated cell type.

Python

```
TRAJECTORY_CLUSTERS = ["Epidermis Cotyledon", "guard cell"]
adata_sub = preview_trajectory_selection(
    adata          = adata,
    clusters       = TRAJECTORY_CLUSTERS,
    annotation_col = ANNOTATION_COL,
    dir_pseudotime = dir_pseudotime,
)
```

Representative output from Section 25:

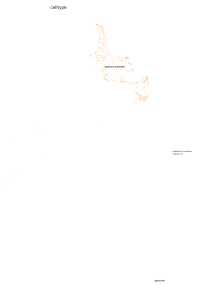


Figure 13: UMAP of the cells selected for the trajectory.

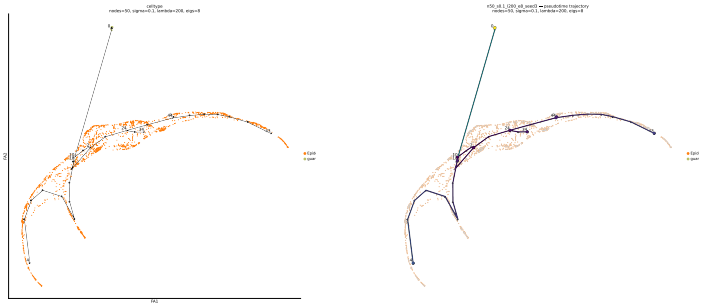
4.4 Section 26 - Trajectory inference

Builds the trajectory with scFates: Palantir diffusion maps, a principal tree, root selection at the progenitor cluster, and pseudotime assignment. Add more `trajectory_run(...)` entries to sweep parameters.

Python

```
ROOT_CLUSTER = "Epidermis Cotyledon"
TRAJECTORY_RUNS = [
    trajectory_run(nodes=50, sigma=0.1, lambda_value=200, eigs=8, seed=3),
]
adata_traj, selected_trajectory_dir, trajectory_runs = run_trajectory_runs(
    adata           = adata,
    clusters        = TRAJECTORY_CLUSTERS,
    root_cluster    = ROOT_CLUSTER,
    annotation_col  = ANNOTATION_COL,
    output_base_dir = dir_pseudotime,
    runs            = TRAJECTORY_RUNS,
)
```

Representative outputs from Section 26:



4.5 Section 27 - Plot genes on trajectory

Plots selected genes on the trajectory layout.

Python

```
GENES = ["AT3G24140", "AT4G21750", "AT4G20260"]
name  = os.path.basename(selected_trajectory_dir)
ensure_expression_in_X(adata_traj)
gene_plots_dir = os.path.join(selected_trajectory_dir, "gene_plots")
os.makedirs(gene_plots_dir, exist_ok=True)
fig = sc.pl.draw_graph(
    adata_traj, color=GENES, title=GENES, show=False, return_fig=True
)
save_plot_18x18(fig, os.path.join(gene_plots_dir, f"{name}_gene_plots.pdf"))
plt.close(fig)
print("SECTION 27 COMPLETE: gene trajectory plots saved")
```

Representative output from Section 27:

4.6 Section 28 - Gene trends

Fits gene expression along pseudotime and saves two heatmaps: the top variable genes, and a custom highlighted set.

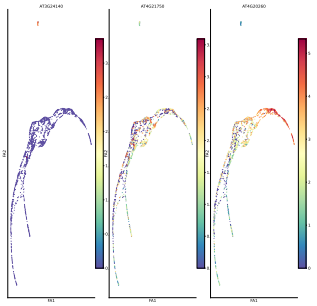
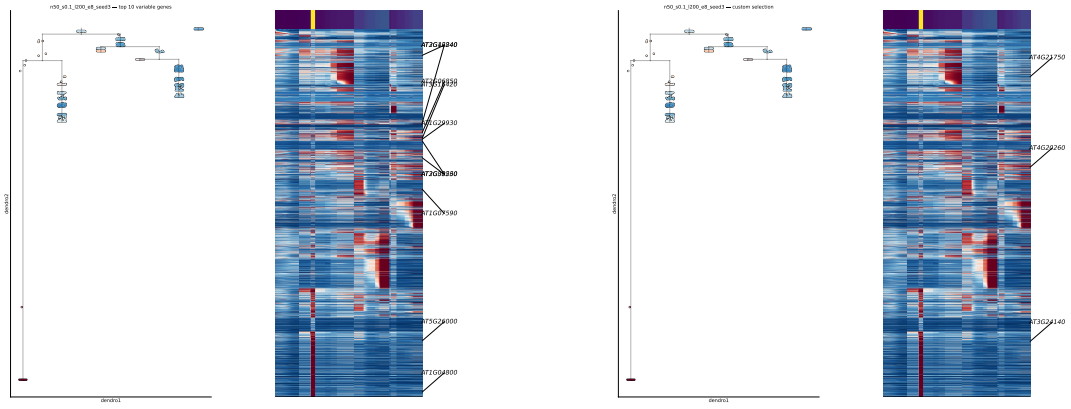


Figure 14: Selected genes on the trajectory layout.

Python

```
TOP_N           = 10
HIGHLIGHT_GENES = ["AT3G24140", "AT4G21750", "AT4G20260"]
ORDERING        = "max"
SPLINE_DF       = 3
top_path, highlight_path = run_step29_gene_trends(
    adata         = adata_traj,
    run_dir       = selected_trajectory_dir,
    name          = os.path.basename(selected_trajectory_dir),
    custom_genes = HIGHLIGHT_GENES,
    top_n         = TOP_N,
    ordering      = ORDERING,
    n_jobs        = N_JOBS,
    spline_df     = SPLINE_DF,
)
print("SECTION 28 COMPLETE: gene trend plots saved")
```

Representative outputs from Section 28 — the top variable genes and the highlighted gene list:



4.7 Section 29 - Final export

Saves the fully processed pseudotime object (trajectory tree, pseudotime, and force-directed layout) as the pipeline's final Chapter 3 output, alongside the curated Chapter 1 object in `resultados/objects/`.

Python

```python
final_h5ad = os.path.join(base_dir, "objects", "pbmc_pseudotime_final.h5ad")
os.makedirs(os.path.dirname(final_h5ad), exist_ok=True)
adata_traj.write_h5ad(final_h5ad)
print(f"SECTION 29 COMPLETE: final pseudotime object saved -> {final_h5ad}")
```

Supplementary Table — Software dependencies and versions

Package versions are those bundled in the `matigara/scrnaseq:latest` Docker image (the public image used for all R and Python steps). R **base-priority** packages are omitted. Cell Ranger is 10x Genomics proprietary software run *outside* the container (Part 1); its version is fixed by the workflow.

Interpreters and command-line tools

Tool	Version
Cell Ranger	9.0.1
R	4.5.3
Python	3.12.3

R packages (421)

Package	Version	Package	Version
abind	1.4-8	locfit	1.5-9.12
anndataR	1.0.2	lubridate	1.9.5
AnnotationDbi	1.72.0	magrittr	2.0.5
ape	5.8-1	maps	3.4.3
aplot	0.2.9	MASS	7.3-65
askpass	1.2.1	Matrix	1.7-5
assertthat	0.2.1	MatrixGenerics	1.22.0
assorthead	1.4.0	MatrixModels	0.5-4
backports	1.5.1	matrixStats	1.5.0
base64enc	0.1-6	memoise	2.0.1
basilisk	1.22.0	mgcv	1.9-4
batchelor	1.26.0	microbenchmark	1.5.0
beachmat	2.26.0	mime	0.13
beeswarm	0.4.0	miniUI	0.1.2
BH	1.90.0-1	minqa	1.2.8
biglm	0.9-3	modelr	0.1.11
Biobase	2.70.0	monocle3	1.4.27
BiocGenerics	0.56.0	nlme	3.1-169
BiocManager	1.30.27	nloptr	2.2.1
BiocNeighbors	2.4.0	nnet	7.3-20
BiocParallel	1.44.0	numDeriv	2016.8-1.1
BiocSingular	1.26.1	openssl	2.4.1
BiocVersion	3.22.0	org.At.tair.db	3.22.0
Biostrings	2.78.0	otel	0.2.0
bit	4.6.0	pak	0.9.5
bit64	4.8.2	parallelly	1.47.0
bitops	1.0-9	patchwork	1.3.2
blob	1.3.0	pbapply	1.7-4
boot	1.3-32	pbkrtest	0.5.5
BPCells	0.3.1	pbmcapply	1.5.1
brew	1.0-10	pheatmap	1.0.13
brio	1.1.5	pillar	1.11.1
broom	1.0.13	pkgbuild	1.4.8
bslib	0.11.0	pkgconfig	2.0.3
cachem	1.1.0	pkgdown	2.2.0
Cairo	1.7-0	pkgload	1.5.2
callr	3.7.6	plotly	4.12.0
car	3.1-5	plyr	1.8.9
carData	3.0-6	png	0.1-9
caTools	1.18.3	polyclip	1.10-7
cellranger	1.1.0	polynom	1.4-1
checkmate	2.3.4	praise	1.0.0
circlize	0.4.18	preprocessCore	1.72.0
class	7.3-23	prettyunits	1.2.0
classInt	0.4-11	processx	3.9.0

Package	Version	Package	Version
cli	3.6.6	profvis	0.4.0
clipr	0.8.1	progress	1.2.3
clue	0.3-68	progressr	0.19.0
cluster	2.1.8.2	promises	1.5.0
clusterProfiler	4.18.4	proxy	0.4-29
clustree	0.5.1	ps	1.9.3
codetools	0.2-20	pscl	1.5.9
colorspace	2.1-2	purrr	1.2.2
commonmark	2.0.0	quadprog	1.5-8
ComplexHeatmap	2.26.1	quantreg	6.1
conflicted	1.2.0	qvalue	2.42.0
corrplot	0.95	R.methodsS3	1.8.2
cowplot	1.2.0	R.oo	1.27.1
cpp11	0.5.5	R.utils	2.13.0
crayon	1.5.3	R6	2.6.1
credentials	2.0.3	ragg	1.5.2
crosstalk	1.2.2	RANN	2.6.2
curl	7.1.0	rappdirs	0.3.4
data.table	1.18.4	rbibutils	2.4.1
DBI	1.3.0	rcmdcheck	1.4.0
dbplyr	2.5.2	RColorBrewer	1.1-3
DelayedArray	0.36.1	Rcpp	1.1.1-1.1
DelayedMatrixStats	1.32.0	RcppAnnoy	0.0.23
deldir	2.0-4	RcppArmadillo	15.2.7-1
Deriv	4.2.0	RcppEigen	0.3.4.0.2
desc	1.4.3	RcppHNSW	0.7.0
DESeq2	1.50.2	RcppML	0.3.7.1
devtools	2.5.2	RcppProgress	0.4.2
diffobj	0.3.6	RcppRoll	0.3.2
digest	0.6.39	RcppTOML	0.2.3
dir.expiry	1.18.0	Rdpack	2.6.6
distributional	0.7.0	readr	2.2.0
doBy	4.7.1	readxl	1.5.0
docopt	0.7.2	reformulas	0.4.4
doParallel	1.0.17	rematch	2.0.0
DOSE	4.4.0	rematch2	2.1.2
dotCall64	1.2	remotes	2.5.0
DoubletFinder	2.0.6	reprex	2.1.1
downlit	0.4.5	reshape2	1.4.5
dplyr	1.2.1	ResidualMatrix	1.20.0
dqrng	0.4.1	reticulate	1.46.0
dtplyr	1.3.3	rhdf5	2.54.1
dynamicTreeCut	1.63-1	rhdf5filters	1.22.0
e1071	1.7-17	Rhdf5lib	1.32.0
ellipsis	0.3.3	RhpcBLASctl	0.23-42
enrichplot	1.30.5	Rhtslib	3.6.0
enrichR	3.4	rjson	0.2.23
eulerr	8.0.0	rlang	1.2.0
evaluate	1.0.5	rmarkdown	2.31
fansi	1.0.7	ROCR	1.0-12
farver	2.1.2	roxygen2	8.0.0
fastcluster	1.3.0	rpart	4.1.24
fastDummies	1.7.6	rprojroot	2.1.1
fastmap	1.2.0	rsample	1.3.2
fastmatch	1.1-8	Rsamtools	2.26.0
fgsea	1.36.2	RSpectra	0.16-2
fields	17.3	RSQLite	3.53.1
filelock	1.0.3	rstatix	0.7.3
fitdistrplus	1.2-6	rstudioapi	0.18.0
FNN	1.1.4.1	rsvd	1.0.5
fontawesome	0.5.3	Rtsne	0.17
fontBitstreamVera	0.1.1	rversions	3.0.0
fontLiberation	0.1.0	rvest	1.0.5
fontquiver	0.2.1	s2	1.1.11
forcats	1.0.1	S4Arrays	1.10.1
foreach	1.5.2	S4Vectors	0.48.1

Package	Version	Package	Version
forecast	9.0.2	S7	0.2.2
foreign	0.8-91	sass	0.4.10
formatR	1.14	ScaledMatrix	1.18.0
Formula	1.2-5	scales	1.4.0
fracdiff	1.5-4	scater	1.38.1
fs	2.1.0	scattermore	1.2
furrr	0.4.0	scatterpie	0.2.6
futile.logger	1.4.9	sctransform	0.4.3
futile.options	1.0.1	scuttle	1.20.0
future	1.70.0	selectr	0.5-1
future.apply	1.20.2	Seqinfo	1.0.0
gargle	1.6.1	sessioninfo	1.2.3
gdtools	0.5.1	Seurat	5.5.0
GeneOverlap	1.46.0	SeuratDisk	0.0.0.9021
generics	0.1.4	SeuratObject	5.4.0
GENIE3	1.32.0	SeuratWrappers	0.4.0
GenomeInfoDb	1.46.2	sf	1.1-1
GenomicRanges	1.62.1	shape	1.4.6.1
gert	2.3.1	shiny	1.13.0
GetoptLong	1.1.1	SingleCellExperiment	1.32.0
ggbeeswarm	0.7.3	sitmo	2.0.2
ggdist	3.3.3	slam	0.1-55
ggforce	0.5.0	slider	0.3.3
ggfun	0.2.0	snow	0.4-4
ggiraph	0.9.6	sourcetools	0.1.7-2
ggnewscale	0.5.2	sp	2.2-1
ggplot2	4.0.3	spam	2.11-4
ggplotify	0.1.3	SparseArray	1.10.10
ggpubr	0.6.3	SparseM	1.84-2
ggraph	2.2.2	sparseMatrixStats	1.22.0
ggrastr	1.0.2	spatial	7.3-18
ggrepel	0.9.8	spatstat.data	3.1-9
ggridges	0.5.7	spatstat.explore	3.8-1
ggsci	5.0.0	spatstat.geom	3.8-1
ggsignif	0.6.4	spatstat.random	3.5-0
ggtangle	0.1.2	spatstat.sparse	3.2-0
ggtree	4.0.5	spatstat.univar	3.2-0
ggvenn	0.1.19	spatstat.utils	3.2-3
gh	1.6.0	spData	2.3.5
gitcreds	0.1.2	spdep	1.4-2
GlobalOptions	0.1.4	speedglm	0.3-4
globals	0.19.1	statmod	1.5.2
glue	1.8.1	stringi	1.8.7
GO.db	3.22.0	stringr	1.6.0
goftest	1.2-3	SummarizedExperiment	1.40.0
googledrive	2.1.2	survival	3.8-6
googlesheets4	1.1.2	svglite	2.2.2
GOSemSim	2.36.0	sys	3.4.3
gplots	3.3.0	systemfonts	1.3.2
graphlayouts	1.2.3	tensor	1.5.1
gridExtra	2.3	tester	0.3.0
gridGraphics	0.5-1	testthat	3.3.2
grr	0.9.5	textshaping	1.0.5
gson	0.1.0	tibble	3.3.1
gtable	0.3.6	tidydr	0.0.6
gtools	3.9.5	tidygraph	1.3.1
h5mread	1.2.1	tidyr	1.3.2
harmony	2.0.3	tidyselect	1.2.1
haven	2.5.5	tidytree	0.4.7
HDF5Array	1.38.0	tidyverse	2.0.0
hdf5r	1.3.12	timechange	0.4.0
hdWGCNA	0.4.11	timeDate	4052.112
here	1.0.2	tinytex	0.59
hexbin	1.28.5	treeio	1.34.0
highr	0.12	tweenr	2.0.3
Hmisc	5.2-5	tzdb	0.5.0

Package	Version	Package	Version
hms	1.1.4	UCell	2.14.0
htmlTable	2.5.0	UCSC.utils	1.6.1
htmltools	0.5.9	units	1.0-1
htmlwidgets	1.6.4	UpSetR	1.4.1
httpuv	1.6.17	urca	1.3-4
httr	1.4.8	urlchecker	1.0.1
httr2	1.2.2	usethis	3.2.1
ica	1.0-3	utf8	1.2.6
ids	1.0.1	uuid	1.2-2
igraph	2.3.2	uwot	0.2.4
impute	1.84.0	vctrs	0.7.3
ini	0.3.1	VennDiagram	1.8.2
IRanges	2.44.0	vipor	0.4.7
irlba	2.3.7	viridis	0.6.5
isoband	0.3.0	viridisLite	0.4.3
iterators	1.0.14	vroom	1.7.1
jquerylib	0.1.4	waldo	0.6.2
jsonlite	2.0.0	warp	0.2.3
kableExtra	1.4.0	WGCNA	1.74
KEGGREST	1.50.0	whisker	0.4.1
KernSmooth	2.23-26	withr	3.0.2
knitr	1.51	wk	0.9.5
labeling	0.4.3	WriteXLS	6.8.0
lambda.r	1.2.4	xfun	0.58
later	1.4.8	xml2	1.5.2
lattice	0.22-9	xopen	1.0.1
lazyeval	0.2.3	xtable	1.8-8
leidenbase	0.1.37	XVector	0.50.0
lifecycle	1.0.5	yaml	2.3.12
limma	3.66.0	yulab.utils	0.2.4
listenv	0.10.1	zellkonverter	1.20.1
littler	0.3.23	zip	2.3.3
lme4	2.0-1	zoo	1.8-15
lmtest	0.9-40		

Python packages (207)

Package	Version	Package	Version
adjustText	1.3.0	notebook	6.5.7
anndata	0.12.16	notebook_shim	0.2.4
annotated-types	0.7.0	numba	0.65.1
anyio	4.13.0	numcodecs	0.16.5
argon2-cffi	25.1.0	numexpr	2.14.1
argon2-cffi-bindings	25.1.0	numpy	2.4.6
array-api-compat	1.14.0	numpy-groupies	0.11.3
arrow	1.4.0	nvidia-cublas	13.1.1.3
asttokens	3.0.1	nvidia-cuda-cupti	13.0.85
async-lru	2.3.0	nvidia-cuda-nvrtc	13.0.88
attrs	26.1.0	nvidia-cuda-runtime	13.0.96
babel	2.18.0	nvidia-cudnn-cu13	9.20.0.48
beautifulsoup4	4.14.3	nvidia-cufft	12.0.0.61
bleach	6.3.0	nvidia-cufile	1.15.1.6
blosc2	4.4.1	nvidia-curand	10.4.0.35
cellbender	0.3.2	nvidia-cusolver	12.0.4.66
certifi	2026.5.20	nvidia-cusparse	12.6.3.3
cffi	2.0.0	nvidia-cusparselt-cu13	0.8.1
charset-normalizer	3.4.7	nvidia-nccl-cu13	2.29.7
click	8.4.1	nvidia-nvjitlink	13.0.88
comm	0.2.3	nvidia-nvshmem-cu13	3.4.5
contourpy	1.3.3	nvidia-nvtx	13.0.85
cuda-bindings	13.3.1	opt_einsum	3.4.0
cuda-pathfinder	1.5.5	packaging	26.2
cuda-toolkit	13.0.2	palantir	1.4.4

Package	Version	Package	Version
cycler	0.12.1	pandas	2.3.3
debugpy	1.8.21	pandocfilters	1.5.1
decorator	5.3.1	parso	0.8.7
defusedxml	0.7.1	patsy	1.0.2
donfig	0.8.1.post1	pexpect	4.9.0
elpigraph-python	0.3.4	pillow	12.2.0
entrypoints	0.4	pip	26.1.2
executing	2.2.1	platformdirs	4.10.0
fa2_modified	0.4	plotly	6.7.0
fast-array-utils	1.4.1	prometheus_client	0.25.0
fastjsonschema	2.21.2	prompt_toolkit	3.0.52
filelock	3.29.0	psutil	7.2.2
fonttools	4.63.0	ptyprocess	0.7.0
fqdn	1.5.1	pure_eval	0.2.3
fsspec	2026.4.0	py-cpuinfo	9.0.0
google-crc32c	1.8.0	pycparser	3.0
h11	0.16.0	pydantic	2.13.4
h5py	3.16.0	pydantic_core	2.46.4
httpcore	1.0.9	Pygments	2.20.0
httpx	0.28.1	pynndescent	0.6.0
idna	3.18	pyparsing	3.3.2
igraph	1.0.0	pyro-api	0.1.2
ipykernel	6.29.5	pyro-ppl	1.9.1
ipython	9.14.0	python-dateutil	2.9.0.post0
ipython-genutils	0.2.0	python-igraph	1.0.0
ipython_pygments_lexers	1.1.1	python-json-logger	4.1.0
ipywidgets	8.1.8	pytz	2026.2
isoduration	20.11.0	PyYAML	6.0.3
jax	0.10.1	pyzmq	27.1.0
jaxlib	0.10.1	referencing	0.37.0
jaxopt	0.8.5	requests	2.34.2
jedi	0.20.0	rfc3339-validator	0.1.4
Jinja2	3.1.6	rfc3986-validator	0.1.1
joblib	1.5.3	rfc3987-syntax	1.1.0
json5	0.14.0	rpds-py	2026.5.1
jsonpointer	3.1.1	rpy2	3.6.7
jsonschema	4.26.0	rpy2-rinterface	3.6.6
jsonschema-specifications	2025.9.1	rpy2-robjects	3.6.5
jupyter	1.1.1	scanpy	1.12.1
jupyter-console	6.6.3	scFates	1.2.4
jupyter-events	0.12.1	scikit-learn	1.9.0
jupyter-highlight-selected-word	0.2.0	scikit-misc	0.5.2
jupyter-lsp	2.3.1	scipy	1.17.1
jupyter_client	7.4.9	scverse-misc	0.0.7
jupyter_contrib_core	0.4.2	seaborn	0.13.2
jupyter_contrib_nbextensions	0.7.0	Send2Trash	2.1.0
jupyter_core	5.9.1	session-info2	0.4.1
jupyter_nbextensions_configurator	0.6.4	setuptools	81.0.0
jupyter_server	2.19.0	shapely	2.1.2
jupyter_server_terminals	0.5.4	simpleppt	1.1.7
jupyterlab	4.5.7	six	1.17.0
jupyterlab_pygments	0.3.0	soupsieve	2.8.4
jupyterlab_server	2.28.0	stack-data	0.6.3
jupyterlab_widgets	3.0.16	statsmodels	0.14.6
kiwisolver	1.5.0	sympy	1.14.0
lark	1.3.1	tables	3.11.1
legacy-api-wrap	1.5	terminado	0.18.1
leidenalg	0.12.0	texttable	1.7.0
llvmlite	0.47.0	threadpoolctl	3.6.0
loompy	3.0.8	tinycss2	1.5.1
lxml	6.1.1	torch	2.12.0
lxml_html_clean	0.4.5	tornado	6.5.6
MarkupSafe	3.0.3	tqdm	4.67.3
matplotlib	3.10.9	traitlets	5.15.0
matplotlib-inline	0.2.2	triton	3.7.0
mellon	1.7.1	typing-inspection	0.4.2

Package	Version	Package	Version
mistune	0.8.4	typing_extensions	4.15.0
ml_dtypes	0.5.4	tzdata	2026.2
mpmath	1.3.0	tzlocal	5.4.4
msgpack	1.1.2	umap-learn	0.5.12
narwhals	2.22.0	uri-template	1.3.0
natsort	8.4.0	urllib3	2.7.0
nbclassic	1.3.3	wcwidth	0.7.0
nbclient	0.10.4	webcolors	25.10.0
nbconvert	6.5.4	webencodings	0.5.1
nbformat	5.10.4	websocket-client	1.9.0
ndindex	1.10.1	widgetsnbextension	4.0.15
nest-asyncio	1.6.0	zarr	3.2.1
networkx	3.6.1		